

Real Life Aggregation – Exploration Notes

1. Difference Between GROUP BY and ORDER BY

- GROUP BY:
 - Used to group rows that have the same values in specified columns into summary rows.
 - Typically used with aggregation functions like SUM(), AVG(), COUNT(), etc.
 - Example: GROUP BY CourseID groups all enrollments by course.
- ORDER BY:
 - Used to sort the final result set by one or more columns.
 - Can sort ascending (ASC) or descending (DESC).
 - Example: ORDER BY AVG(Rating) DESC sorts average ratings from highest to lowest.

2. Why Use HAVING Instead of WHERE With Aggregates?

- WHERE: Filters rows before grouping or aggregation.
 - You can't use WHERE with aggregate functions like AVG() or SUM().
- HAVING: Filters groups after aggregation.
 - Allows conditions on aggregated values, e.g., HAVING AVG(Rating) > 3.

3. Common Mistakes Beginners Make With Aggregation Queries

- Using WHERE instead of HAVING for aggregate filters.
- Forgetting to include non-aggregated columns in GROUP BY.
- Misunderstanding NULL values in aggregates.
- Assuming aggregate functions ignore duplicates (when COUNT does not).
- Using aggregates without GROUP BY and expecting grouped results.

4. When to Use COUNT(DISTINCT ...), AVG(...), SUM(...) Together

- To analyze unique counts (COUNT DISTINCT), average values (AVG), and total sums (SUM) simultaneously.
- Example: In a sales report, count distinct customers, average sale amount, and total revenue per region.
- Useful for detailed summarization and multi-dimensional analysis.

5. Performance Impact of GROUP BY and How Indexes Help

- GROUP BY can be resource-intensive on large datasets due to sorting and aggregation.
- Indexes on grouping columns can speed up the grouping operation by allowing faster data access.
- Proper indexing can reduce I/O and improve query execution plans.
- However, too many indexes may slow down writes (INSERT/UPDATE).
- Use EXPLAIN plans to analyze and optimize queries with GROUP BY.